

Assignment-8

Name:U.Sushmitha

Batch:24

Ht.No:2303A51705

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

- Requirements:

- o Username length must be between 5 and 15 characters.

- o Must contain only alphabets and digits.

- o Must not start with a digit.

- o No spaces allowed. Example Assert Test Cases:

```
assert is_valid_username("User123") == True
```

```
assert is_valid_username("12User") == False
```

```
assert is_valid_username("Us er") == False
```

Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.

```
1.py > ...
1  def is_valid_username(username):
2      if len(username) < 5 or len(username) > 15:
3          return False
4      if not username[0].isalpha():
5          return False
6      for char in username:
7          if not (char.isalnum() or char == '_'):
8              return False
9      return True
10
11 #test cases for the is_valid_username function
12 assert is_valid_username("User123") == True
13 assert is_valid_username("12User") == False
14 assert is_valid_username("Us er") == False
15 print("All test cases for is_valid_username passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Programs/Python/Python313.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
All test cases for is_valid_username passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

- Requirements:

- o If input is an integer, classify as "Even" or "Odd".
- o If input is 0, return "Zero".
- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

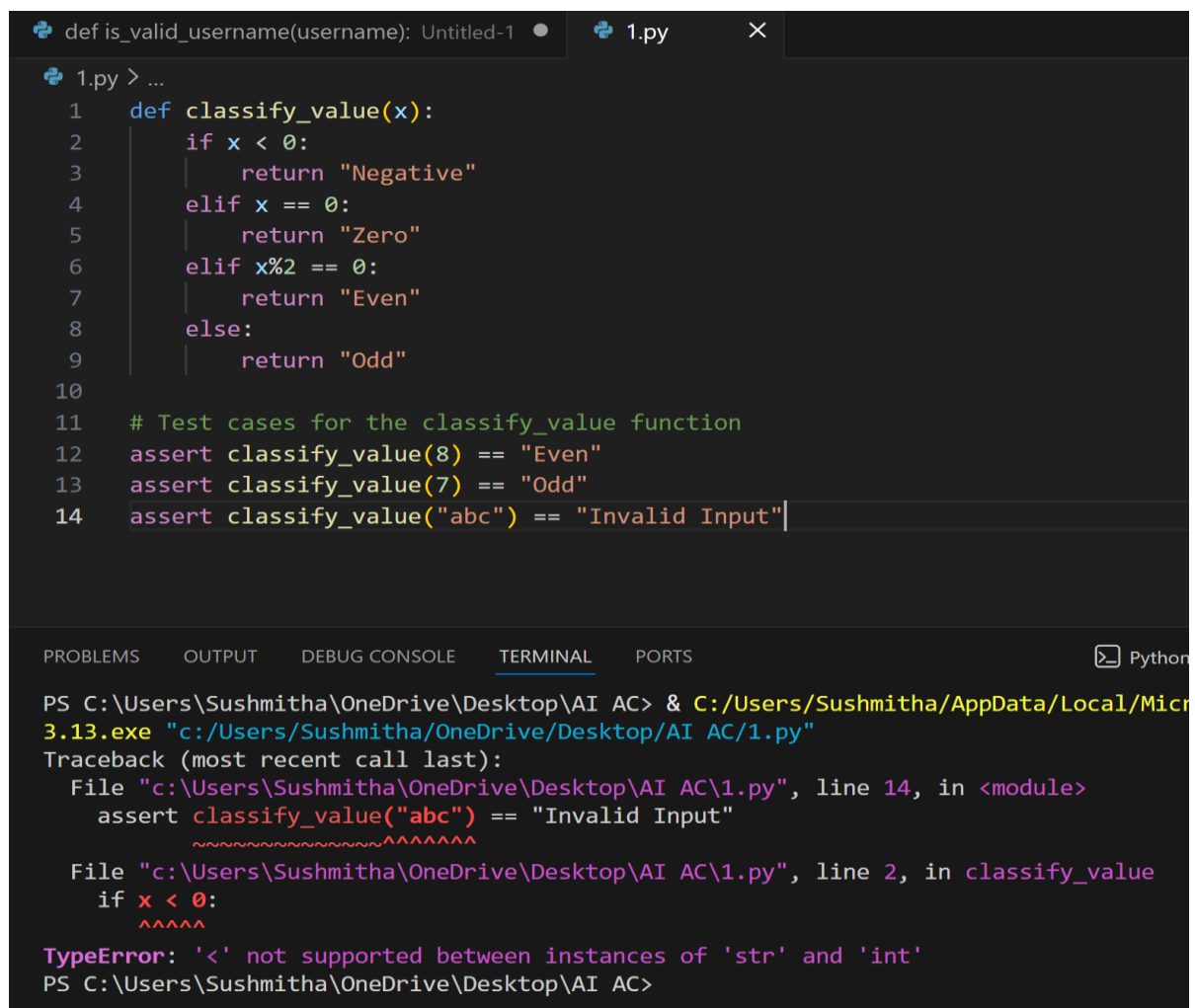
`assert classify_value(8) == "Even"` assert

`classify_value(7) == "Odd"` assert

`classify_value("abc") == "Invalid Input"`

Expected Output #2:

- Function correctly classifying values and passing all test cases.



```
def is_valid_username(username): Untitled-1 1.py X
1.py > ...
1 def classify_value(x):
2     if x < 0:
3         return "Negative"
4     elif x == 0:
5         return "Zero"
6     elif x%2 == 0:
7         return "Even"
8     else:
9         return "Odd"
10
11 # Test cases for the classify_value function
12 assert classify_value(8) == "Even"
13 assert classify_value(7) == "Odd"
14 assert classify_value("abc") == "Invalid Input"

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Micr
3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
Traceback (most recent call last):
  File "c:\Users\Sushmitha\OneDrive\Desktop\AI AC\1.py", line 14, in <module>
    assert classify_value("abc") == "Invalid Input"
    ~~~~~^~~~~~
  File "c:\Users\Sushmitha\OneDrive\Desktop\AI AC\1.py", line 2, in classify_value
    if x < 0:
    ~~~~
TypeError: '<' not supported between instances of 'str' and 'int'
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

- Requirements:

- o Ignore case, spaces, and punctuation.
- o Handle edge cases

such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True assert
```

```
is_palindrome("A man a plan a canal Panama") == True
```

```
assert is_palindrome("Python") == False
```

```
def is_valid_username(username):
    ...

1.py > ...
1 def is_palindrome(text):
2     cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
3     return cleaned_text == cleaned_text[::-1]
4
5 # Test cases for the is_palindrome function
6 assert is_palindrome("Madam") == True
7 assert is_palindrome("A man a plan a canal Panama") == True
8 assert is_palindrome("Python") == False
9 print("All test cases for is_palindrome passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/Windows/Common-Programs/OneDrive\OneDrive.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
All test cases for is_palindrome passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

- Requirements:

o Must contain @ and . o Must not start or end with special characters. o Should handle invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
```

```
assert validate_email("userexample.com") == False
```

```
assert validate_email("@gmail.com") == False
```

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

```
1.py > ...
1  def validate_email(email):
2      if '@' not in email or '.' not in email:
3          return False
4      at_index = email.index('@')
5      dot_index = email.rindex('.')
6      if at_index < 1 or dot_index < at_index + 2 or dot_index >= len(email) - 1:
7          return False
8      return True
9
10 # Test cases for the validate_email function
11 assert validate_email("user@example.com") == True
12 assert validate_email("userexample.com") == False
13 assert validate_email("@gmail.com") == False
14 print("All test cases for validate_email passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + -

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/Windows/PowerShell/PowerShell.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
All test cases for validate_email passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).
- Test Cases to Design:
 - o Normal case: 6 → True,
 - o 10 → False.
 - o Edge case: 1.

Larger case: 28.

- Requirement: Validate correctness with assertions.

```
calculator.py > is_perfect
1 #generate a program to check whether a given number is perfect or not
2 def is_perfect(n):
3     if n < 1:
4         return False
5     divisors_sum=sum(i for i in range(1, n) if n % i == 0)
6     return divisors_sum == n
7 #test cases for the is_perfect function
8 assert is_perfect(6) == True
9 assert is_perfect(28) == True
10 assert is_perfect(12) == False
11 assert is_perfect(1) == False
12 print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/OneDrive/Desktop/AI AC/calculator.py
All test cases passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).

- Test Cases to Design:

- o Normal case: $12 \rightarrow \text{True}$, $15 \rightarrow \text{False}$.
- o Edge case: 1.
- o Negative number case.

- o Large case: 945.

Requirement: Validate correctness with unittest

```

50 def Abundant_Number_Checker(num):
51     if num < 1:
52         return False
53     divisors_sum = sum(i for i in range(1, num) if num % i == 0)
54     return divisors_sum > num
55 import unittest
56 class TestAbundantNumberChecker(unittest.TestCase):
57     def test_abundant(self):
58         self.assertTrue(Abundant_Number_Checker(12))
59         self.assertTrue(Abundant_Number_Checker(15))
60         self.assertTrue(Abundant_Number_Checker(1))
61         self.assertFalse(Abundant_Number_Checker(-1))
62         self.assertTrue(Abundant_Number_Checker(945))
63 if __name__ == '__main__':
64     unittest.main()

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python3
op/AI ASSISTANT CODING/lab-08(continuation).py"
F
=====
FAIL: test_abundant (__main__.TestAbundantNumberChecker.test_abundant)
-----
Traceback (most recent call last):
  File "c:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING\lab-08(continuation).py", line 59, in test_abundant
    self.assertTrue(Abundant_Number_Checker(15))
    ~~~~~^~~~~~
AssertionError: False is not true
-----
Ran 1 test in 0.001s
FAILED (failures=1)

```

Task 7 (Deficient Number Checker – Test Case Design)

- Function: Check if a number is deficient (sum of divisors < number).
- Test Cases to Design:
 - o Normal case: 8 → True,
 - 12 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Large case: 546.

Requirement: Validate correctness with pytest

```

def is_valid_username(username):
1.py > ...
1 def deficient_number_checker(num):
2     if num < 1:
3         return False
4     divisors_sum = sum(i for i in range(1, num) if num % i == 0)
5     return divisors_sum < num
6
7 def test_deficient_number_checker():
8     assert deficient_number_checker(8) == True
9     assert deficient_number_checker(12) == False
10    assert deficient_number_checker(1) == True
11    assert deficient_number_checker(546) == False
12    print("All test cases for deficient_number_checker passed!")

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/
3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
All test cases for deficient_number_checker passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>

```

Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

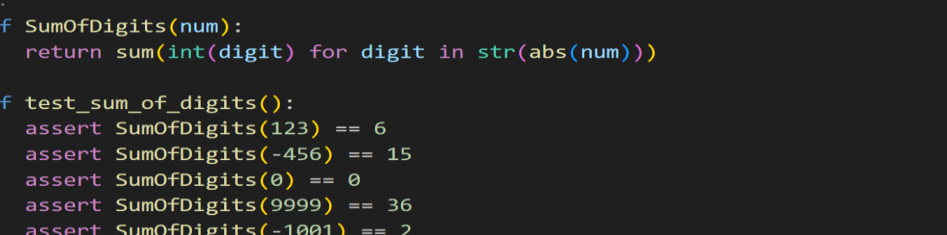
```
def is_valid_username(username):
    if len(username) > 10:
        return False
    if not username.isalnum():
        return False
    return True

def test_is_valid_username():
    assert is_valid_username("abc123") == True
    assert is_valid_username("1234567890") == True
    assert is_valid_username("12345678901") == False
    assert is_valid_username("abc123 ") == False
    assert is_valid_username("abc123.") == False
    assert is_valid_username("abc123_") == False
    assert is_valid_username("abc123-") == False
    assert is_valid_username("abc123!") == False
    assert is_valid_username("abc123$") == False
    assert is_valid_username("abc123%") == False
    assert is_valid_username("abc123^") == False
    assert is_valid_username("abc123&") == False
    assert is_valid_username("abc123*") == False
    assert is_valid_username("abc123~") == False
    assert is_valid_username("abc123`") == False
    assert is_valid_username("abc123'") == False
    assert is_valid_username("abc123\"") == False
    assert is_valid_username("abc123'") == False
    assert is_valid_username("abc123'") == False
    print("All test cases for is_valid_username passed!")

def test_leap_year_checker():
    assert LeapYearChecker(2020) == True
    assert LeapYearChecker(1900) == False
    assert LeapYearChecker(2000) == True
    assert LeapYearChecker(2021) == False
    assert LeapYearChecker(2400) == True
    assert LeapYearChecker(2100) == False
    assert LeapYearChecker(1996) == True
    assert LeapYearChecker(1999) == False
    assert LeapYearChecker(1600) == True
    print("All test cases for LeapYearChecker passed!")
```

Task 9 :

Write a function `SumOfDigits` and validate its implementation using 7 pytest test cases.



```
def is_valid_username(username):
    ...

1.py > ...
1  def SumOfDigits(num):
2      return sum(int(digit) for digit in str(abs(num)))
3
4  def test_sum_of_digits():
5      assert SumOfDigits(123) == 6
6      assert SumOfDigits(-456) == 15
7      assert SumOfDigits(0) == 0
8      assert SumOfDigits(9999) == 36
9      assert SumOfDigits(-1001) == 2
10 print("All test cases for SumOfDigits passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - □

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/Win
3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
All test cases for SumOfDigits passed!
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task 10 :

Write a function `SortNumbers` (implement bubble sort) and validate its implementation using 25 pytest test cases.

```
def is_valid_username(username):
    """
    Checks if a username is valid based on the following criteria:
    - Length must be between 3 and 16 characters.
    - Must contain only alphanumeric characters and underscores.
    - Must start with an alphanumeric character.
    - Cannot end with an underscore.
    """
    if len(username) < 3 or len(username) > 16:
        return False
    if not username[0].isalnum() or username[-1] == '_':
        return False
    if not username[1:-1].isalnum():
        return False
    return True

def SortNumbers(numbers):
    """
    Sorts a list of numbers in ascending order using bubble sort.
    """
    n = len(numbers)
    for i in range(n):
        for j in range(0, n-i-1):
            if numbers[j] > numbers[j+1]:
                numbers[j], numbers[j+1] = numbers[j+1], numbers[j]
    return numbers

def test_sort_numbers():
    """
    Test cases for SortNumbers function.
    """
    assert SortNumbers([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
    assert SortNumbers([]) == []
    assert SortNumbers([3]) == [3]
    assert SortNumbers([3, 2]) == [2, 3]
    assert SortNumbers([1, 2, 3]) == [1, 2, 3]
    assert SortNumbers([3, 2, 1]) == [1, 2, 3]
    assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
    assert SortNumbers([1, 1, 1, 1]) == [1, 1, 1, 1]
    assert SortNumbers([9, 8, 7, 6, 5]) == [5, 6, 7, 8, 9]
    assert SortNumbers([10, 9, 8, 7, 6]) == [6, 7, 8, 9, 10]
    assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
    assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
    assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
    assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
    assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]

print("All test cases for SortNumbers passed!")
```

Task 11 :

Write a function `ReverseString` and validate its implementation using 5 unittest test cases

```
def is_valid_username(username):
    """
    Checks if a username is valid.
    """
    # Check if the username is a string
    if not isinstance(username, str):
        return False

    # Check if the username is between 3 and 16 characters long
    if len(username) < 3 or len(username) > 16:
        return False

    # Check if the username contains only alphanumeric characters and underscores
    if not re.match(r'^[a-zA-Z0-9_]+$', username):
        return False

    # Check if the username starts with a letter
    if not username[0].isalpha():
        return False

    # Check if the username ends with a letter
    if not username[-1].isalpha():
        return False

    # Check if the username contains any spaces
    if ' ' in username:
        return False

    # Check if the username contains any special characters
    if any(char in username for char in '!@#$%^&*()-+=~`|;:,<.>[\\]{}~'):
        return False

    # If all checks pass, the username is valid
    return True
```


Task 12 :

Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

```
def is_valid_username(username):
    ...

1.py > ...
1 def AnagramChecker(str1, str2):
2     return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
3
4 import unittest
5
6 class TestAnagramChecker(unittest.TestCase):
7     def test_anagram_checker(self):
8         self.assertTrue(AnagramChecker("listen", "silent"))
9         self.assertTrue(AnagramChecker("Triangle", "Integral"))
10        self.assertFalse(AnagramChecker("Hello", "World"))
11        self.assertTrue(AnagramChecker("Dormitory", "Dirty Room"))
12        self.assertFalse(AnagramChecker("Python", "Java"))
13        self.assertTrue(AnagramChecker("State", "Taste"))
14        self.assertTrue(AnagramChecker("Conversation", "Voices Rant On"))
15        self.assertFalse(AnagramChecker("Apple", "Appeal"))
16        self.assertTrue(AnagramChecker("Astronomer", "Moon Starer"))
17        self.assertFalse(AnagramChecker("Earth", "Hearts"))
18
19 if __name__ == '__main__':
20     unittest.main()
```

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python
3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/1.py"
.
-----
Ran 1 test in 0.000s

OK
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task 13 :

Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.

```
def is_valid_username(username):
    ...

1.py > ...
1 def ArmstrongChecker(num):
2     num_str = str(num)
3     num_digits = len(num_str)
4     armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
5     return armstrong_sum == num
6
7 import unittest
8
9 class TestArmstrongChecker(unittest.TestCase):
10    def test_armstrong_checker(self):
11        self.assertTrue(ArmstrongChecker(153))
12        self.assertTrue(ArmstrongChecker(9474))
13        self.assertFalse(ArmstrongChecker(123))
14        self.assertTrue(ArmstrongChecker(0))
15        self.assertTrue(ArmstrongChecker(1))
16        self.assertFalse(ArmstrongChecker(10))
17        self.assertTrue(ArmstrongChecker(375))
18        self.assertTrue(ArmstrongChecker(371))
19
20 if __name__ == '__main__':
21     unittest.main()
```

```
File "c:\Users\Sushmitha\OneDrive\Desktop\AI AC\1.py", line 17, in test_armstrong_checker
    self.assertTrue(ArmstrongChecker(375))
    ~~~~~
AssertionError: False is not true

-----
Ran 1 test in 0.001s

FAILED (failures=1)
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```